



Information Systems Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Control of Flexible Software Development Under Uncertainty

Michael L. Harris, Rosann Webb Collins, Alan R. Hevner,

To cite this article:

Michael L. Harris, Rosann Webb Collins, Alan R. Hevner, (2009) Control of Flexible Software Development Under Uncertainty. Information Systems Research 20(3):400-419. <http://dx.doi.org/10.1287/isre.1090.0240>

Full terms and conditions of use: <http://pubsonline.informs.org/page/terms-and-conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2009, INFORMS

Please scroll down for article—it is on subsequent pages



INFORMS is the largest professional society in the world for professionals in the fields of operations research, management science, and analytics.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Control of Flexible Software Development Under Uncertainty

Michael L. Harris

School of Business, Indiana University Southeast, New Albany, Indiana 47150, harris60@ius.edu

Rosann Webb Collins, Alan R. Hevner

Information Systems and Decisions Sciences, College of Business, University of South Florida,
Tampa, Florida 33620 {rcollins@coba.usf.edu, ahevner@coba.usf.edu}

When should software development teams have the flexibility to modify their directions and how do we balance that flexibility with controls essential to produce acceptable outcomes? We use dynamic capabilities theory and an extension of control theory to understand these questions. This work is examined in a case study. Our results demonstrate that flexibility may be needed when the starting conditions are uncertain and that effective control in these situations requires use of traditional controls plus a new type of control we term emergent outcome control.

Key words: control theory; uncertainty; flexible development; agile development; emergent outcome control

History: Brian Fitzgerald, Senior Editor and Associate Editor. This paper was received on June 1, 2007, and was with the authors 19 months for 3 revisions. Published online in *Articles in Advance* August 25, 2009.

1. Introduction

It is Saturday night in Metropolis. Three couples are out for an evening of entertainment. The Ables head to the theater district to see the latest revival of Hamlet. Although they have seen Shakespeare's tragedy performed a number of times, they look forward to a fresh interpretation from an up-and-coming director and the eclectic cast he has assembled. Meanwhile, the Bakers travel to an intimate playhouse where a critically acclaimed group of actors are performing improvisation. The Bakers enjoy the unscripted spontaneity of improv theater. The actors will play-off of the audience to shape the performance in a way that delivers a uniquely personal experience. Finally, the Charles' attend a performance art festival in the city park. Artists perform on platforms scattered throughout the venue. The couple is thrilled by the creativity and the passion of the artists; although, they have to admit that they do not "get" several of the pieces. All three couples arrive home buoyed by their respective artistic experiences.

Software development is a creative endeavor, not unlike the performing arts in many ways. Austin and Devin (2003) explore similarities between the arts and the artful making activities of business, in particular, software development. One insight they offer is that

software artifacts are similar to artistic performances in the way that they emerge from an iterative creative process that must be carefully managed to converge to a successful result.

A central dynamic in software development is the trade-off between control and flexibility. Strong management control is desirable in environments with strict budgetary or schedule constraints and those that require high-quality results, such as safety-critical systems. This control is typically achieved through upfront development of a plan that defines system requirements, architectures, designs, budgets, and schedules with the hope that the development will proceed smoothly based on the predetermined script. In executing the plan, the developers have limited flexibility to modify their activities within the plan's controls. The analogy to a theatrical play is apt. Script and stage directions provide broad controls within which the director and actors have some range to interpret the vision and meanings of the playwright.

A movement to encourage flexibility in software projects has resulted in a number of development approaches, such as rapid prototyping (Baskerville and Stage 1996), synchronize and stabilize (Cusumano

and Yoffie 1999), Scrum (Schwaber and Beedle 2002), eXtreme programming (XP) (Beck and Andres 2004), and the rational unified process (RUP) (Kruchten 2000). The essence of flexibility is the ability to improvise in reaction to changes—changes in requirements, budgets, schedules, risks, etc. Improvisation can be defined as the degree to which the planning and execution of an action converge in time (Moorman and Miner 1998). Thus a flexible software development team can be related to an improv theater group in the process of real-time problem solving. Similarly to improv acting, the developers coordinate with one another, abide by rules of development (e.g., design and programming guidelines, security and assurance guarantees), and produce a coherent result. Prior product development research has linked improvisation with creativity and innovation (Weick 1998, Eisenhardt and Tabrizi 1995).

In addition to the plan-driven and more flexible development approaches, an idiosyncratic, ad hoc development approach has always been prevalent in the software field. Termed an “adhocracy” by Mintzberg and McHugh (1985), such a development organization would be characterized by (1) dynamic and complex development environments requiring one-of-a-kind outputs, (2) highly skilled experts deployed in temporary teaming arrangements, (3) coordination of efforts by semiformal structures (i.e., direct supervision and standardization are discouraged), and (4) power over decisions is diffused based on the availability of information and expertise. The analogy here would be to performance art, which is highly creative, produces individual expert outputs, and requires significant effort to integrate into coherent themes (Bowman 2008).

How then are we to study and understand the relationships of control and flexibility in software projects? Some researchers describe project agility using dimensions such as technology sophistication, project size, staffing expertise, diversity of stakeholders, and application domains (Boehm and Turner 2004). Others highlight the need to tailor development methods to projects (Fitzgerald et al. 2003, 2006). However, there is a scarcity of formal theory to describe the balance between control and flexibility for a software development project.

In this research, we compare three development paradigms. Of particular interest is controlled-flexible development defined as a development approach that expects the development team to be flexible (improvise) but that also establishes controls to ensure that team decisions are congruent with organizational objectives. This controlled-flexible approach is compared with two more traditional paradigms: plan-driven development and ad hoc development. Plan-driven development specifies most decisions before development begins and the team is expected to largely adhere to the scripted plan. Ad hoc development is an organic process that asks the developers to self-monitor to achieve organizational goals. This paper begins with a theoretical analysis to understand when and how a controlled-flexible approach may be used and subsequently presents an exploratory field study based on in-depth interviews with practicing software managers and developers.

2. Grounding Theories

Our analysis is grounded in two theories. First, the dynamic capabilities theory explains *when* flexible development is needed; when market and technological uncertainty are high (Teece et al. 1997, Eisenhardt and Martin 2000). As uncertainty increases, firms may allow development teams to reconfigure their operational processes to help “design, manage the development of, and launch the new products” (Pavlou and El Sawy 2006, p. 202). Next, we turn to control theory (Ouchi 1977, 1979, 1980; Kirsch 1997) to understand *how* controlled-flexible development works. Controlled-flexible development uses traditional control mechanisms, but it also requires a new control category, emergent outcome control, to explain unique flexible control processes.

2.1. Theory of Dynamic Capabilities

The resource-based theory of the firm is a strategic theory that assumes a firm’s advantage comes from its unique assets and capabilities (Barney 1996). Management will organize themselves around unique assets and capabilities and will seek out new assets and capabilities to gain an advantage. However, in cases of “rapid technological change and fast-shifting market circumstances” (Teece et al. 1997, p. 512) it may not be possible to plan for the future.

If unpredictability is high, even risk-management (Lyytinen et al. 1998) may be ineffective. Risk management requires the ability to foresee potential outcomes and to make decisions based on expected value. In high velocity (Eisenhardt 1989a) environments, it may be difficult to enumerate possible outcomes. Instead of preplanning, the organization may build a decentralized capability to learn about changes and to react quickly (Teece et al. 1997). Eisenhardt and Martin (2000) discuss rules at Intel that allowed operational management to reallocate corporate resources well before senior managers recognized a transition was occurring.

Much of the foundational work on dynamic capabilities comes from the product development literature. Early product development research prescribed a controlled, plan-driven process, but more recently scholars have suggested more flexible alternatives (Krishnan and Ulrich 2001). Peters and Waterman (1982, p. 119) recommend an iterative, learning approach when they coin the phrase, “Ready-fire-aim.” This approach involves a quick trial followed by an evaluation, retargeting, and follow up trials. Takeuchi and Nonaka (1986) suggest the metaphor of a rugby scrum that involves complex, coordinated team interactions.

Other researchers suggest a fast-moving, interactive development approach to explore markets and test technology alternatives (Bhattacharya et al. 1998, Bourgeois and Eisenhardt 1988, Brown and Eisenhardt 1997, Dahan and Mendelson 2001, Eisenhardt 1989a, Eisenhardt and Tabrizi 1995, Iansiti 1995, Karagozoglu and Brown 1993, MacCormack and Verganti 2003, Harris et al. 2007). This body of research emphasizes the difficulty of developing plans up front when market conditions are chaotic and when technology risk is high. Successful organizations use dynamic capabilities to reconfigure their approach as they learn and adjust to changes.

The relationships between uncertain environments, dynamic capabilities, and new product success are detailed and tested by Pavlou and El Sawy (2006) in their investigation of how a firm’s ability to effectively use IT functionality influences competitive advantage in new product development. They find that a firm’s dynamic capability (the ability to reconfigure resources by effectively sensing the environment,

learning, coordinating, and integrating interaction patterns) has a significant, positive impact on work units’ operational processes that design, manage the development of, and launch new products.

In this research, we apply these findings to the world of software development by drawing on the correspondence between product and software development. Increasingly, this correspondence is direct because many development teams work on software products, including shrink-wrapped software, enterprise resource planning systems, and Internet-based services. Even custom software development has similarities with product development (Nambisan 2003, Nambisan and Wilemon 2000). Furthermore, custom development can be influenced by the dynamic forces affecting other firm competencies (Wheeler 2002).

The argument for applying dynamic capabilities to software development can also be made logically. The essence of dynamic capabilities is that high-velocity (Eisenhardt 1989a) environments require the ability to reconfigure on-the-fly (Teece et al. 1997, Eisenhardt and Martin 2000) because change cannot be predicted. This change can be driven by market uncertainty or technical uncertainty (Teece et al. 1997). Even if a development project does not serve a market, we would argue that the conflicting needs of multiple classes of stakeholders, organization changes, and priority changes can create an organizational-version of market uncertainty. Furthermore, teams often face technical uncertainty in their first experiences with new technology (e.g., “the silver bullet syndrome,” McConnell 1996, p. 314).

One study goal is to identify the circumstances under which flexible or adaptive software development is needed (Lyytinen and Rose 2006). Different methods may be appropriate for different software development contexts (Fitzgerald et al. 2006). According to dynamic capability theory, turbulent environments (unpredictable needs and fast-changing technology) signal a need for dynamic capabilities. In contrast, a stable environment would suggest the use of a plan-driven approach, as epitomized by the highly scripted, waterfall development approach.

2.2. Control Theory

Dynamic capabilities theory explains when organizations need flexible processes, but it is less forthcoming

on how those processes work. Traditionally, flexibility was achieved through an organic approach (Ouchi 1980, Robey 1996) that removes barriers and relies on skills and semiformal structures (Mintzberg and McHugh 1985). Yet, in their study of adaptive processes, Eisenhardt and Tabrizi (1995, p. 108) state: "But playful, fluid organic processes fail to capture the importance of focus and structure that emerges here." To study how dynamic capabilities work, we turn to *control theory*, which is a study of the mechanisms that can be used to achieve organizational objectives.

In his seminal work on control, Ouchi (1977, 1979, 1980) describes three types of controls. *Output control* is used when desired outcomes can be established a priori and actual accomplishments can be measured against the a priori targets. *Behavioral control* depends on the existence of specific behaviors that can be reliably linked to successful results. In behavioral control, management must be able to observe behaviors-in-use and must be able to measure deviations from expected behaviors. A third type of control is an informal control called *clan control*. Clan control depends on socialization, training, and selection to achieve broad goal congruence between workers and the organization. It then relies on those workers to determine the best solution for the good of the organization. Clan control relies on self control and subtle peer-to-peer signals rather than on formal control by legitimate authority. Clan control is the basis for an organic, loosely-coupled organization (Ouchi 1980).

Control theory is a powerful paradigm that offers insights into management's ability to focus employees on organizational objectives. Its findings are strongest when it involves deterministic tasks. Output control sets detailed specifications for outcomes. Behavioral control assumes a cause and effect relationship between specific behaviors and desired objectives. Clan control asks employees to chart their own course. It assumes that employees share goals with the organization, and that they will work in the organization's best interest.

When uncertainty exists, control theory suggests the use of clan control (Ouchi 1979), and indeed many studies find instances of clan control in use (e.g., Ouchi and Johnson 1978, Orlikowski 1991, Kirsch 1997, Cardinal et al. 2004). However, studies

of environments with high ambiguity have shown less reliance on pure clan control than control theory would suggest (Eisenhardt and Tabrizi 1995, Eisenhardt and Sull 2001, Harris et al. 2007) and one study indicates that practitioners may be uncomfortable relying on organic controls (Kirsch 1997).

Several investigations extend the use of control theory into software development environments in which a portfolio of controls may be required (Choudhury and Sabherwal 2003, Henderson and Soonchul 1992, Kirsch 1997, Nidumolu and Subramani 2003–2004, Orlikowski 1991). Researchers have examined controls in dynamic settings (Cardinal et al. 2004, Choudhury and Sabherwal 2003, Kirsch 2004, Orlikowski 1991). In these studies, the term *dynamic* refers to the way that priorities shift through a lifecycle along with the control changes that occur as priorities change. Another form of dynamism is studied by Nidumolu and Subramani (2003–2004), who identify *low decentralization of performance criteria*, in which criteria used to evaluate project outcomes (i.e., outcome controls) vary from case to case when there is high uncertainty. Although they posited that *low decentralization of performance criteria* would result in lower performance, their empirical results do not support this hypothesis, and they suggest that the operation of controls in this kind of setting is more complex and needs further exploration.

However, we are still left with a theoretical gap in our understanding. Even as modified by the work on control portfolios and in dynamic settings, control theory still suggests that the deterministic output and behavior controls will be insufficient when ambiguity and uncertainty are high (Ouchi 1979) and that clan control may not be a sufficient control mechanism on its own (Eisenhardt and Tabrizi 1995, Kirsch 1997). Thus there is a need for a new form of control to explain controlled-flexible development.

2.3. Emergent Outcome Controls

One way to conceptually transform control theory to encompass controlled-flexible development is to study the knowledge embedded in flexible development practices and synthesize those findings using the existing control theory framework (Harris et al. 2009). We can start by analyzing the control implications of the agile manifesto (Beck et al. 2001).

The agile manifesto dictates working software and flexible interactions/collaborations during development. *Working software* refers to software that can be used or demonstrated during development, not just at the final software release. The ability to continuously demonstrate the software as it emerges from development allows the team to rapidly adjust the software direction. In the terminology of control theory, software is an output of the development process; however, this focus on emergent software is quite different from what we would expect in traditional output control.

Traditional output control is enacted in two steps. The first step involves creation of an a priori specification of the requirements, quality metrics, budgets, and schedules (Nidumolu and Subramani 2003–2004) early in the development process.¹ After development completes, the second step evaluates the delivered outputs for compliance with the a priori specification. Traditional output control in software development does not speak to the way in which outcomes are determined in agile methods, where they emerge from an iterative development process.

Flexible development begins with a partial specification. Even the most flexible team must know whether they are creating an accounting system or a video game. Constraints may also include architecture plans, interface descriptions, and resource limitations. However, a flexible specification leaves many key questions unanswered. A flexible specification acts as a scope boundary (Harris et al. 2009) not as a detailed description. It constrains the feasible solution set. Thus as Table 1 reveals, traditional output control focuses on evaluation of results, whereas emergent outcome control steers development.

In summary, this conceptual analysis proposes a new type of control, emergent outcome control, that more accurately describes how controlled-flexible processes work. Whereas traditional output control evaluates the final output, this new control steers the evolution of the output. Emergent outcome control

Table 1 Comparison of Output Control and Emergent Outcome Control

	Purpose	Frequency	Evaluator	Construction of standard	Comparison
Output	Evaluation	Once	Supervisor	A priori	Completed project versus specification
Emergent outcome	Corrective action	Continuous	Multiple stakeholders	Evolving by stakeholder	Emergent outcomes versus tacit specifications

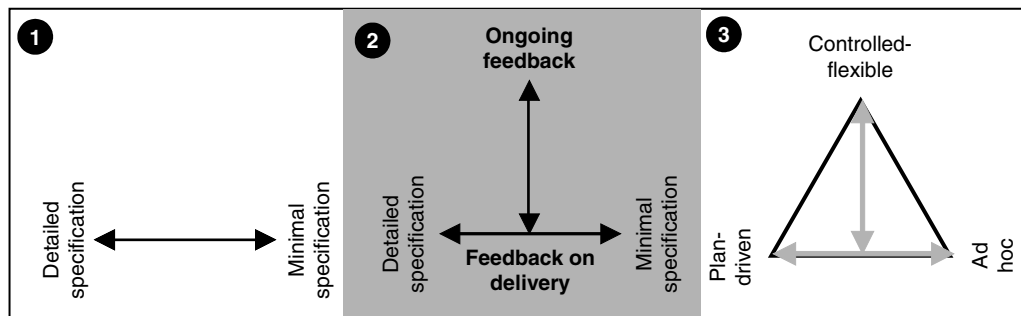
is also different from behavioral control. Behavioral control specifies the transformative behaviors that produce output. Emergent outcome control does not speak to the how the software is created; it constrains and guides the development process. To be clear, we do not claim that flexible development methods use only emergent outcome controls but will include such controls in a broader control portfolio that may include output and behavioral controls in addition.

In our analysis of flexible development approaches we find two primary emergent outcome control mechanisms: (1) scope boundaries and (2) ongoing feedback. Scope boundaries constrain the set of feasible solutions without dictating specific outcomes. These boundaries can be thought of as risk management mechanisms that shape the attention of the software team (Lyytinen et al. 1998). Examples from XP include a shared vision, partial specifications, predefined architectures, fixed application programming interfaces (APIs), and resource constraints. The intersection of all the scope boundary controls defines the feasible space for exploration.

Teams with few boundaries need more feedback. XP has broad boundaries but it contains pervasive feedback including pair programming, daily team review meetings, colocation of team members with visible progress indicators, daily software builds, colocation with customer representatives, and very short release cycles to gain broad market exposure. In contrast, the RUP approach tightly defines development scope and only offers feedback through iteration releases (Harris et al. 2009). If scope boundaries are tightened sufficiently to remove all choice from the development team, there is no need for interim feedback and the approach becomes more of a specification-driven approach. If scope boundaries

¹ In the traditional waterfall development approach, detailed behavioral controls also appear in the project plan, which lists specific, required tasks, such as testing and documentation. During the project as well as at the end of the project, the development team may be evaluated on their compliance with these required behavioral controls.

Figure 1 Software Development Approaches



are broad and there are no opportunities for feedback until the final release, then the development process will resemble an ad hoc approach. Thus, the goal of a controlled-flexible development method is to achieve a trade-off among emergent outcome controls; balancing the restrictiveness of scope boundaries with opportunities for dynamic feedback.

3. Research Model and Hypotheses

To initiate our research study, we ran an exploratory focus group with experienced developers to discuss issues of control and flexibility in software development. Invariably, we found confusion among the various terms used to classify development. In clarifying these terms, we began by noting our focus on uncertainty and change as described by the dynamic capabilities model. Our focus was on the way that development methods address environmental change. In Figure 1, we illustrate the fundamental differences between three archetypes of development approaches, which we term *controlled-flexible*, *plan-driven*, and *ad hoc*.

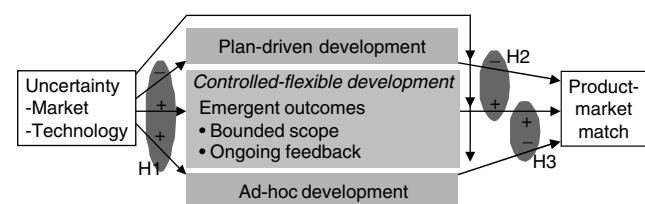
Controlled-flexible development approaches are defined as those that rely on emergent outcome controls: scope boundaries and ongoing feedback. Referring to Figure 1, scope boundaries are shown on the *x*-axis as the variation in the amount of specification that occurs early in development. The amount of ongoing feedback is shown on the *y*-axis.

As scope boundaries become especially tight, they begin to resemble detailed a priori specifications, but as individual boundaries are loosened or removed they begin to resemble broad mission statements. The other dimension to emergent outcome control is the

presence of ongoing feedback. Both the plan-driven approach and the ad hoc approach only require feedback on delivery. However, controlled-flexible approaches require ongoing feedback to apply corrective action to the decisions made by the development team. These two axes define the space of potential methods. A pure plan-driven approach only requires a clear, detailed specification. As long as everyone agrees on the output there is no need to check on progress. A pure ad hoc approach has no detailed checks. The team is given a broad goal and is expected to work on that goal until they achieve the desired result. A controlled-flexible approach will use scope boundaries to define the allowable space for exploration and it will use significant amounts of feedback to check on decisions as they are made throughout the development process. We note that pure examples are unlikely in practice and that any given instance of development is likely to be located on the interior of this triangular space. This definition of controlled-flexible development leads to the research model in Figure 2.

Our research hypotheses, based on the dynamic capabilities and control theories, are highlighted in the model. Dynamic capabilities theory supports

Figure 2 Research Model



the link between the nature of a firm's market and technology environment and its capabilities (in this case, software development approach), as well as the impact of the fit between environment and development on product-market match (how well the software developed meets market requirements). Control theory provides the conceptual basis for understanding the differences between the various development methods: plan-driven, controlled-flexible, and ad hoc.

Prior research on dynamic capabilities shows that as firms' environments become more uncertain, there is more need for flexibility; "rapidly creating situation-specific new knowledge" (Eisenhardt and Martin 2000, p. 1111). The very definition of dynamic capabilities is "the firm's ability to integrate, build, and reconfigure internal and external competencies to address rapidly changing environments" (Teece et al. 1997, p. 516). These competencies are "embedded in existing organizational routines, structure, and processes" (O'Reilly and Tushman 2008, p. 188). Environmental turbulence stems from market uncertainty (consumer needs, or competitor strategies) and technological uncertainty (developments and breakthroughs) (Jap 2001).

The relationship between the amount of environmental turbulence (market or technology uncertainty, or both) and organization processes was found in a study of 72 product development projects in the computer industry by Eisenhardt and Tabrizi (1995). They found that in the moderately-dynamic main-frame sector development processes were structured, linear, and planned, whereas in higher velocity sectors of the market, simple, experiential, and iterative processes were found. This relationship between the firm's environment and processes is shown in Table 2.

As we apply this theory to the process of software development, plan-driven development approaches are an example of the detailed, linear, and stable processes, and flexible and ad hoc development approaches are examples of simple, iterative approaches that rely on quick adaptation to the specific situation. Therefore we posit that the existence of market or technological, or both, uncertainty in a firm's environment makes a plan-driven approach to software

Table 2 Relationship Between Market Environment and Firm Processes

	Moderately dynamic market environment	High-velocity market
Industry structure	Stable	Ambiguous
Market boundaries	Well defined and; players clearly identified	Blurred, with players ambiguous and shifting
Market competitors	Clearly identifiable with stable or slowly changing strategies	Ambiguous and shifting; new and unexpected strategies
Market change	Linear and predictable consumer needs	Nonlinear and unpredictable consumer needs
Technology	Slowly evolving	Changing, with breakthroughs, perhaps disruptive
Firm processes		
Pattern	Detailed, analytic routines that rely extensively on existing knowledge	Simple, experiential routines that rely on newly created knowledge specific to the situation
Execution	Linear	Iterative
Stable	Yes	No

Adapted from Eisenhardt and Martin (2000, p. 1115).

development impractical and leads to a flexible development approach. Thus:

HYPOTHESIS 1 (H1). *As the market and technology become more uncertain, software development is less likely to be managed using a plan-driven method.*

Given a choice between development methods we must consider whether the method chosen is successful. How do we define success of a method? One market may require leading edge features (Baskerville et al. 2001) whereas another may emphasize reliability and performance. A central feature of a dynamic market is the inability to define exact market needs a priori. Eisenhardt and Martin (2000) suggest that the success of a solution should be measured by how well the solution matches the needs of its specific target environment. Therefore, we determine success as the relative match between the development initiative and the market's needs.

Empirical evidence suggests a relationship between success and the fit of a firm's processes to the uncertainty in its environment (O'Reilly and Tushman 2008). For example, Verona and Ravasi (2003) found that the success of a Danish hearing aid company

was based on the firm's ability to implement new technological advances in new organizational processes and structures. In contrast, Tripsas and Gavetti (2000) showed that although Polaroid had developed new digital imaging technologies, the rigidity of the company's processes and failure to implement a new business model resulted in failure in the new market. Therefore, our expectation is that a team with dynamic capabilities will be better able to meet the needs of a changing market. Thus:

HYPOTHESIS 2 (H2). *In an uncertain environment, a controlled-flexible software development method is more likely to produce a product that matches the market's needs than a plan-driven method.*

Of course an ad hoc development team can also be flexible. The problem is that there is no mechanism to align the team with the market. Developers may gold-plate the product, they may misunderstand the market, they may produce components that do not integrate into a coherent solution, or they may take inappropriate shortcuts (McConnell 1996). Research indicates that successful teams use more structured approaches than the purely organic ad hoc approach (Eisenhardt and Tabrizi 1995, Eisenhardt and Sull 2001, Harris et al. 2007). As discussed in §2, there is a distinction between ad hoc and controlled-flexible software development based on the use of emergent outcome controls. Scope boundaries provide structure to the development effort, and ongoing feedback is a key control mechanism to align what the team is developing with market needs. Thus:

HYPOTHESIS 3 (H3). *In an uncertain environment, a controlled-flexible software development method is more likely to produce a product that matches the market's needs than an ad hoc approach.*

4. Research Design

4.1. Exploration via Structured Interviews

The initial research model was discussed in a pilot focus group with four experienced software developers. The discussion supported the rationale of the underlying model but suggested that measurement through a survey instrument would be impractical for several reasons. First, clear definitions for terms such as "plan-driven," "controlled-flexible,"

and "ad hoc" are not generally accepted. Second, the control mechanisms used in projects are varied and sometimes tacit. Tacit controls are difficult to recall without prompts and it would be difficult to provide an exhaustive list of prompts on a survey instrument for the wide variety of control mechanisms. These factors led to the use of interview-based data gathering research design. A typical interview began with broad questions to allow developers to offer evidence of unforeseen ideas. For example, there was a general question about peer interactions. However, after interviewees were asked exploratory questions, they were also probed via very specific questions, such as, "Do you regularly conduct code reviews?"

In the conducted interviews, selected software managers and developers described their experiences in software product development. Case studies are frequently used in the study of organizational controls and dynamic environments that are known to be idiosyncratic and context dependent (Eisenhardt and Martin 2000). Case studies are especially appropriate when how and why questions are posed (Yin 1994). In this situation the relevant concerns are how development methods are controlled and why organizations choose to use flexible methods. In addition, a case study approach is useful because controls are expected to be context dependent.

The steps to achieve rigor in positivistic case research are identified by Dubé and Paré (2003). These steps were applied in this study except for the proposed test of competing theories. In this instance, there are no directly competing theories available. The closest argument would be that flexible approaches are chosen in response to time pressures instead of in response to uncertainty. In fact, an ad hoc team saves on overhead and may be the fastest if they can deliver a satisfactory solution. However, time pressures do not stand the test of logic in the case of the controlled-flexible approaches. Multiple iterations with rework and constant reengineering are likely to take longer than an approach that delivers accurately the first time. In our data collection and analysis we were sensitive to the potential influence of time pressures, but we did not feel it was sufficiently challenging to treat it as a competing theory. We needed to provide another means to falsify our prediction. Thus, the interview guide specifically explores counterexamples with questions such

as “Can you think of a project with high uncertainty that was managed using a detailed plan?”

As recommended by Eisenhardt (1989b), theoretical sampling was used to select the participants in the study. Study sites were screened based on their experience with flexible development. For initial screening purposes we looked for organizations that began development without complete plans. Our goal was to study multiple instances of flexible development to allow replication and extension of the dynamic capabilities and control theories that underlay the study (Huber and Van de Ven 1995). Within screened organizations we sought a range of projects that allowed us to provide saturation of the constructs in the study. The unit of analysis is the individual software project. In most cases, projects had multiple informants at multiple levels. This allowed triangulation because we could compare a manager’s viewpoint with that of a developer. The description of the four organizations and seven projects is shown in Table 3.

Most of the interviews were performed in one company, a large application service provider (ASP) with over 300 developers. This allowed us to compare

projects while holding the organizational environment constant. Within this organization an initial meeting with the directors was used to identify projects and individuals that would prove most useful to testing the study hypotheses. We also interviewed four team members on a project at a small software business. This project had struggled over two years with several starts and stops. In the process it had transitioned through several different development approaches. Within one project and a single team this gave us access to multiple development methods.

Two additional interviews were also performed. These interviews did not follow the interview protocols and they were not used for hypothesis testing. However, they did provide rich data that allowed us to elaborate on the study findings. One interview was with the chief technical officer (CTO) of a small mobile device software company that used offshore developers in Russia. Another interview was a short discussion with a web development company that used a plan-driven approach in an environment that most others approached using flexible techniques.

Table 3 Organizations and Software Development Projects in the Study

1. <i>Large ASP</i> —This company offers centralized transaction and reporting services to a distributed network of business customers. About 300 developers work in teams of 3 to 6 people. Programs exchange data through shared data repositories and interact through public API’s. Four teams were interviewed.	
Custom project team—Three Interviews (Four people)	This team is working on a custom version of existing software for a new international client. Because one interview included two subjects, there were actually four interviewees.
Reporting project team—Three Interviews	The interviewees were team leads on different projects. Projects included new products and enhancements. Some reports were custom designed for individual clients; others were designed for general market use.
Maintenance project team—Four Interviews	This team managed enhancement requests to existing products. These requests could include either defect correction or the addition of new functionality.
Consumer product project team—Four Interviews	This consumer-orientated product was developed in response to a sudden market change. There was a short window of opportunity for product introduction.
2. <i>Small software company</i> —A four person team builds software to support operations for a vertical industry. The company has 2 existing modules in use by about 1,000 small business customers.	
Operations product project team—Four Interviews	They have been working on a new module for two years. At the time of the interviews the product was well overdue, and they were attempting their third development method.
Uncoded interviews—used for exploratory and confirmatory evidence	
3. Mobile device software company—One Interview	Uses offshore outsourcer to deliver in a flexible process. Data collected through handwritten notes. CTO interviewed. This was not coded because there was only one subject, and the notes were handwritten not audio recordings.
4. Web development company—One Interview (with two people)	This small company delivers web products using a plan-driven approach. They provided time for a short interview to describe why they didn’t use a flexible approach, but it did not follow the full interview protocol and it was not coded.

Table 4 Coding Definitions

Coders categorized each interviewee's utterance based on these category definitions, with one or several codes. To allow for the emergence of alternative explanations, these definitions are intentionally broad. For example, in coding utterances for success or failure, coders were asked to mark both discussions of the hypothesized variable (meeting the customers' needs to assess product-market match) and other ways of defining success or failure.

Category	Choose transcript excerpts that . . .
Success/failure	Discuss indications of success or failure. Specifically to include how well the software meets customer needs, meets milestones, bugs, customer satisfaction, and any others.
Controls/constraints	Factors that control, guide, or constrain developer. Anything that affects what or how a developer programs. Also, excerpts that show no controls. Examples of controls include written plans, time limits, roles, peer feedback, . . . (20 examples listed).
Methods	Discuss specific methods such as XP. Discuss categories such as plan-driven, controlled-flexible, and ad hoc.
Factors that influence the method chosen	Look for statements like this: We used a flexible approach because . . .
Uncertainty—Market or technical	This category may overlap the previous category. It specifically asks the coders to pay attention to statements of uncertainty and the impact of uncertainty on the method chosen, the controls used, and on the outcomes.

4.2. Measurement and Data Collection

Interviews were guided by an outline to elicit unanticipated evidence as well as to test specific hypotheses. To support elaborative coding (Auerback and Silverstein 2003), we began with open-ended questions. Subsequently, the questions became specific to directly probe hypotheses. Initially the interview focused on the subject's current project. Next, subjects were asked to identify a contrasting reference project from their past experience. If the current project was flexible, subjects were asked to contrast it with a plan-driven or an ad hoc project. Whereas the current projects had multiple informants, the contrast projects did not. However, the contrast comparisons were all relative to the current project. In addition, the interviewer probed for counterexamples that might falsify the proposed model. For example, a subject might be asked to identify a project with high uncertainty that used a plan-driven approach.

Although an ideal case study would involve multiple forms of data collection, in this instance all data came from interviews. Instead of deeply probing into an individual project, the decision was made to pursue a broader range of projects. This provided a data set that encompassed more conditions and allowed for more opportunity to falsify the proposed hypotheses and to offer boundary conditions to the model. Because the data were gathered as relative comparisons between projects and were triangulated between subjects, this approach yielded sufficient redundancy.

Interviews from the five multi-interview projects were audiotaped, transcribed, edited for anonymity, and coded. The data reduction included three phases: (1) coding to categorize statements from the transcripts, (2) rating to analyze the coded excerpts for the project development method, and (3) consolidation to compare and analyze findings. The coders were provided with written category definitions (see Table 4) but they were blind to the study hypotheses. Each transcript was independently coded by two separate coders. The independent work was then compared and reconciled. As Table 5 shows, the agreement on all transcripts was acceptable (Cohen's kappa at or above 0.70, except for one transcript at 0.69).

In the second phase of data reduction, two independent raters used the coded statements to measure project development method. Both of the raters were experienced software professionals as well as trained academics. The raters examined the coded constructs to determine the types of control exhibited. For example, Table 6 shows the evidence from one interview.

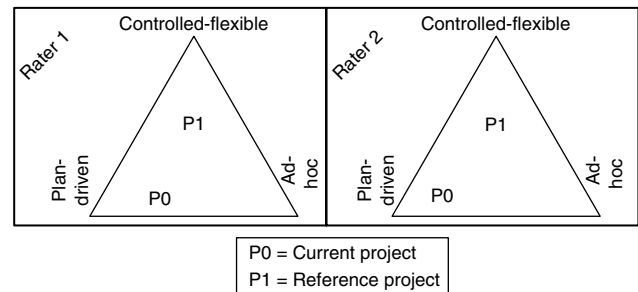
Both raters felt the evidence in Table 6 demonstrated a controlled-flexible environment, not an ad hoc environment. Along the continuum between controlled-flexible and plan-driven, they placed it nearer to controlled-flexible, although they judged it to be about 20% of the way toward plan-driven. The evidence in Table 6 shows significant occurrence of both scope boundaries and feedback mechanisms. Another interview with a junior developer on the

Table 5 Intercode Agreement for Each Transcript

Interviewee	No. of passages coded	Percent of agreement (%)	Cohen's kappa
<i>Large ASP—Custom project</i>			
Manager and lead	58	83	0.82
Senior developer	71	80	0.80
Developer	53	72	0.71
<i>Large ASP—Reporting project</i>			
Lead	52	87	0.86
Lead II	74	78	0.78
Lead III	60	82	0.81
<i>Large ASP—Maintenance project</i>			
Director	42	83	0.83
Senior manager	63	81	0.80
Senior developer	82	79	0.79
Developer	51	78	0.78
<i>Large ASP—Consumer product project</i>			
Director	26	73	0.72
Manager	70	70	0.69
Technical lead	122	80	0.80
Junior developer	72	71	0.70
<i>Small software company—Operations product project</i>			
Manager	88	75	0.74
Senior developer	58	81	0.80
Developer	94	78	0.77
Junior developer	54	72	0.71

Table 6 Evidence of Emergent Outcome Controls (ASP-Consumer Manager Interview)

Examples of scope boundaries	Examples of feedback mechanisms
- Establish a high level design that locks down over 50% to 75% of the requirements before development begins. - Architecture and technology platform established up front. - The design is anchored by the need to interface with six preexisting external systems. - Junior developers are assigned more stable tasks and work under the guidance of senior developers. Senior developers have experience with the corporate standards for each task area (i.e., User Interface, Back-end, ...). - Multiphase development with milestone reviews between phases. This limits the change in a phase. Early phases involve more stable components. Later, flexible phases are anchored to this stable base. - A small team with defined roles limits resources and change during each iteration.	- Small teams whose members talk to each other. - Team leader role to coordinate progress. - Daily software builds force continuous integration. - Business analysts review system before integration testing. - Delivery in phases with feedback at each delivery point. - Customers involved in feedback process. - Management stakeholders review throughout process. - Senior developers review work of junior developers. - Multiple levels of testing. Independent tester sits on team. Assists in unit testing and transfers unit level knowledge between components. External testing looks at integration issues and occurs after initial release by team.

Figure 3 Comparative Ratings for One Interview, Current, and Reference Projects

same project resulted in data favoring more of a plan-driven approach. This may be because junior developers are more subject to purely plan-driven controls. Furthermore, an upfront plan that dictates up to 75% of requirements provides a greater level of specification that is found in most flexible methods.

The rating process did require judgment. Therefore the raters independently placed each interview on a project map (e.g., Figure 3). In this example, the current project is labeled P0, and a previous project from the same subject is labeled P1. The raters met after their independent work to discuss rating differences. As illustrated in Figure 3, small differences between raters were common; however, the findings were sufficiently similar to classify the techniques.

The final data reduction step involved triangulation and summarization among team members. Team member ratings were consolidated into a composite map (Figure 4). The oval maps allowed us to determine whether different team members experienced different control environments. In general, the consolidated maps (Table 7) revealed consistency within teams.

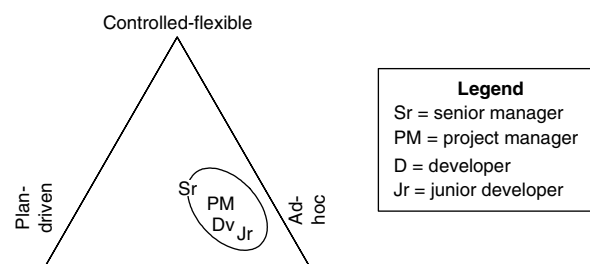
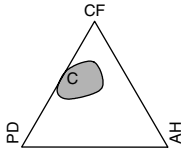
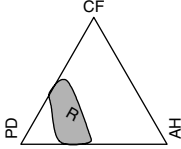
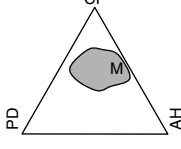
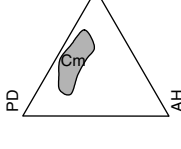
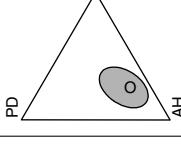
Figure 4 Composite Map in the Current Project (Small Software Company)

Table 7 Evidence for Methods Used in Current Projects

Large ASP Custom 	Controlled-flexible—plan-driven combination (CF/PD) This team is building a custom solution for a business outside the United States. The developers have more application experience than the customer. The developers are running as controlled a process as possible. They are predocumenting requirements and managing to a strict set of controls. However, there are undocumented country-specific legacy interface issues. Trial and error is the only way to uncover these interface requirements.
Reporting 	Primarily plan-driven (PD) The interviewees are lead developers on different reporting applications. They develop specifications for themselves and for junior developers. The method is fairly plan-driven. Most requirements are known a priori. A few details emerge in a controlled-flexible or ad-hoc fashion. However, if the source system is under development with an emergent outcome approach, then the report group is forced to become more flexible because the API and data definitions may be in flux until the end of the project.
Maintenance 	Flexible development (between controlled-flexible and ad hoc) (CF/AH) This team uses flexible development methods—between controlled-flexible and ad hoc. Where possible, controls are established. However, rather than plan and control changes, the developers are more likely to try them out, and iterate through multiple market tests. The director and manager lean toward the left side of the composite map, and the developers are toward the right side. However all four ratings are still relatively close, and there is not enough separation to distinguish a significant difference.
Consumer 	Plan driven—controlled-flexible (PD/CF) This team is building a consumer-orientated product for a new market. They are trying their best to plan the product upfront, but it is a market that is new to the world, and no one really understands the market's needs. About 25% of the upfront plan is changed by the time the product is implemented. Additionally, the group uses an iterative approach with many small point releases to help manage the change cycle.
Small software company—Operations product 	Ad-hoc development (AH) This organization is ad hoc. The controls that exist are ceremonial (Ouchi 1980). This case illustrates that a portfolio of controls need to reinforce each other: If customer feedback, ticketing process, and Quality Control (QC) process were integrated, then the map would lean toward the controlled-flexible area. As it is, these mechanisms are isolated—it's as if someone tried to build a dam by scattering sticks in the water.

5. Results

The results from the interview data analyses provide strong support for the three hypotheses; however the results also reveal boundary conditions. The study further elucidates the proposed constructs and the ways these constructs work in practice. As a result, this work makes contributions to research and practice.

5.1. Development Methods in Use

The research hypotheses focused on links between uncertainty and method choice (H1) and between method choice and product-market match (H2 and H3). Based on the coding and rating of the interview transcripts, the methods used in the multi-interview

projects are identified in Table 7. The results show that most projects used a combination of methods. In addition, the diagrams reveal that there are differences in the control environment perceived by team members on the same project. This variation is represented by the diagram shape and is discussed in the comments column in Table 7. The controls identified in the project vary slightly based on the seniority level and development role of the individual.

5.2. Hypothesis 1—Selection of Development Methods

The first hypothesis (H1) examines the effect of uncertainty on the type of control mechanism. The find-

Table 8 Support for H1

ASP-custom Manager and lead (strong support) Sr. developer (strong support) Developer (strong support)	↑(Market uncertainty) → ↑(Controlled-flexible) ↓(plan-driven) The transcripts included evidence for the hypothesized relationship. One interview placed a higher stress on the role of market uncertainty because the technical side was more known. Also the method chosen might differ between individuals: an individual with a more junior role may be given more predictable, plan-driven tasks. Alternative Explanation: There was evidence that uncertainty is only one reason for choosing a method; an alternative reason mentioned was selection based on time pressure.
ASP—reporting Lead (strong support) Lead II (strong support) Lead III (strong support)	↓(Technology/market uncertainty) → ↑(plan-driven) Strong support for increased use of plan-driven methods when technology and market uncertainty are both low. There was also support for the differing methods for coworkers based on their roles/levels. Some higher technology uncertainty: This team runs plan-driven projects, but it is sometimes tripped up because another team is feeding the data to this team and is running a more flexible approach. As a result, the other team's flexibility creates uncertainty in the technical environment for this team. This can lead to less reliance on a plan-driven method.
ASP-maintenance Director (weaker support) Sr. manager (strong support) Sr. developer (strong support) Developer (support)	↑ Market uncertainty → ↑ flexible (between controlled-flexible & ad hoc) General support. The director interview had weaker support simply because it did not have enough detail to completely verify the findings. The transcript also emphasized how dependencies on other areas can affect the methods in the current area: too much flexibility in a dependent area may cause high uncertainty in the current project. Alternative explanation: Evidence a method could be chosen based on time pressure.
ASP-consumer Director (N/A) Manager (Strong Support) Tech Lead (Support) Developer (Support)	↑ Market uncertainty → ↑(controlled-flexible) ↓(plan-driven) General support. There was not enough comparison evidence from the director to test H1, but there was support from the other interviews. Counterexample: The manager was asked if he/she had worked in a plan-driven project with high uncertainty. The manager identified a project at another company. Although the existence of this project contradicted the first hypothesis, it validated the overall model because it failed after 2.5 years and almost \$25 million.
Small co.-operations product Manager (Some support) Sr. Developer (Strong support) Developer (Strong support) Jr. Developer (Support)	↑ Market uncertainty → ↑(ad hoc) General support for the findings. Code recorded a partial miss because one organization chose a plan-driven approach even when uncertainty existed. This counterexample occurred because the company based their choice on the existence of a standard methodology at the company instead of on the characteristics of the project. Alternative explanation: Ad hoc chosen when projects are small.
Web development	Counterexample: ↑ market uncertainty → ↑ plan-driven Not coded (nonstandard interview format), but it includes evidence of a counterexample. This company uses a plan-driven approach to build web sites for third parties. The determining factor is not uncertainty—it's the need to make a profit on a fixed price contract.

ings (Table 8) are consistent with the hypothesis: an increase in uncertainty leads to a more flexible approach. The support is consistent and strong for H1 based on each informant's current project. Additional support comes from the interviewees' comparison of the current project to a reference project from their past experience. They consistently referred to the level of uncertainty in the reference projects. An illustrative comment was:

The controlled-flexible approach...was probably driven by...the inability to get current customer requirements because the whole industry was entering a new facet. The one that was plan-driven, we had a much better idea...well actually, it was an existing product that we were, you know, about to make a major enhancement to.

Technology uncertainty also decreases the chance of using a plan-driven approach. Thus:

Interviewer: "Compared to the current [plan-driven] project...what was the reason for that [controlled-flexible] project being managed differently?"

Subject: "...It is because [the controlled-flexible one] was dealing with new technology"

In addition to real experiences, subjects were asked for opinions on hypothetical project situations. Once again, H1 was supported with general preferences for more controlled-flexible approaches as uncertainty increased.

Interviewer: "So, if you have uncertainty, would it lead you to use a plan-driven or not use a plan-driven [method]?"

Subject: “My personal preference is probably [to] use ad hoc or a controlled-flexibility method together.”

The interview process also used a critical incident approach to uncover counterexamples of the hypothesized relationship between the environment and development method. For example, interviewees were asked if they had ever used a plan-driven approach when there was uncertainty. The data uncovered through these questions proved useful in exploring exceptions and boundary conditions. One common exception occurred when there is no option for selecting other than the organization’s standard development method. The manager for the operations product team in the small software company described a situation at a previous company in which a structured waterfall approach was used because it was the standard approach. The outcome in the referenced project was poor because the plan-driven approach could not handle the uncertainty in that project environment.

A second exception is that the choice of method can be constrained by the methods used in coordinating development units. For example, one organization tested a highly iterative development process designed to gain feedback as the product evolved. However, the interim output stalled with the quality control group. Quality control never converted to the iterative process and they only began testing when the final release was completed. As a result, no one outside the immediate development group saw the interim releases. Similarly, an organization that usually used plan-driven approaches had to be more flexible when their interface systems were in development by a group using a flexible approach. A development team cannot simply implement a new method without the buy-in and active cooperation of all coordinating development units. These exceptions may be particularly prevalent in distributed settings in which separate development units may be methodologically as well as physically distant.

5.2.1. Development Time Pressures. Whereas the study hypotheses focus on the role of uncertainty in selecting a development method, the importance of time pressure was revealed in several interviews.

A typical example:

Interviewer: So, could you have somehow waited to start development until they knew more of what the details were?

Response: No,...it was going to go live...as mandated by [the government] we had to have our product ready to collect data...

Pressure to meet a market entry date with minimum features may reduce the desire to achieve an optimal product-market match. These groups used different development approaches than those in the current study. The following excerpt illustrates the approach used.

...[management says] oh by the way, we need this in 35 days. we have methods to try and do that kind of procedurally, but if, at the extreme what you start doing is really just, is balancing risk with quality then and cutting corners. ...there are certain [times] where you have to break out of the box and kind of just get something done.

The effect of time pressures on methodology choice is far from simple. If uncertainty is low, a plan-driven approach may be the quickest way to complete a project. With extreme time pressure, it may be impossible to complete a plan, but a controlled-flexible approach will not be faster. A controlled flexible approach involves a search process and feedback that can result in rework, and thus can slow delivery. If there is not time for a plan-driven approach, and not enough time for the rework of a controlled-flexible approach, the team must accept increased delivery risk.

Development trade-offs with time pressure are as complex as those under uncertainty. However, we do not consider this an invalidation of the uncertainty-based model of development but instead the discovery of a boundary condition. This reinforces the need for a portfolio of controls instead of a single set of control choices. If the primary issue facing a project team is uncertainty and the goal is to match the market’s needs, then the team can choose controlled-flexible methods. If the primary issue is time pressure, other controls may be appropriate. Although the current study indicates the methods that might be used in conditions of uncertainty, future studies can be used to establish the appropriate methods for other starting conditions.

5.2.2. Project Size. Another boundary condition indicated by the study is the project size. This was indicated by a junior developer on the operations software team. Ad hoc teams are more likely to be used for small projects. In fact, in the case of extremely small projects, an ad hoc approach may be chosen irrespective of uncertainty. For a short-duration project or for a very small number of developers it may be less costly to simply build the software and iterate through changes.

5.3. Hypotheses 2 and 3—Relationship of Development Methods and Product-Market Match

The study results reveal that the best approach for matching the software product to the market's needs in conditions of uncertainty is the controlled-flexible approach. For example, in the following instance the organization had a standard approach that they used, irrespective of the project's uncertainty, with poor product-market match results:

...the last project [at my last company] was...a new service offering that was being developed and it was, you know, going through our normal, at the time, our normal methodology—develop requirements, you

know, a very structure[d] waterfall approach. But the organization lacked a lot of the subject matter expertise... It was identified as a risk,...but, we kept marching ahead...they ended up building the wrong thing. ...We weren't successful because we had to rewrite it again after we were done.

In the same transcript, the subject described how a team that was more flexible could work with its customers to handle change during the project instead of waiting until the end; thus delivering a good solution. The support for H2—in uncertain environments, the controlled-flexible approach is more likely to produce a product-market match than a plan-driven method—is summarized in Table 9.

The model also predicts that a controlled-flexible approach will do a better job than an ad hoc approach and this finding is also supported by the data. However, again we found some boundary conditions. For example, an ad hoc failure is illustrated in the following excerpt:

We have one ad hoc that's not going to make it now. ...the person who was working on it was new. And she was asking me how we do the ad hoc method. So, ...with the amount of delays and the lack of confidence with that deliverable, we had to pull it

Table 9 Support for H2

ASP-custom Manager and lead (N/A) Sr. developer (N/A) Developer (N/A)	Support: N/A This group did not support or weaken the argument for H2. There was not enough evidence from this group to make a determination either way. In most cases, the comparison projects were ad hoc, and in the cases where the comparisons were plan-driven, the evidence was not sufficient to compare the product-market match against a controlled-flexible approach.
ASP-reporting Lead (support) Lead II (strong support) Lead III (strong support)	↑ Uncertainty + plan-driven → ↓ product-market match There was solid evidence that plan-driven approaches failed as uncertainty increased. There was also evidence that controlled-flexible approaches did match the market. However, the examples referred to the plan-driven approach, and the higher success of the controlled-flexible approach was implied, but not backed up with details.
ASP-maintenance Director (weak support) Sr. manager (strong support) Sr. developer (support) Developer (weak support)	↑ Market uncertainty + controlled-flexible → ↑ product-market match In two cases, the significance of the support was lower. The director proved to be weak in the details so the evidence was less convincing. There is also ambiguity for the developer, but the description left room for interpretation as to the degree of control of one project, therefore casting some doubt on how success was achieved.
ASP-consumer Director (N/A) Manager (support) Tech lead (weak support) Developer (N/A)	Weak support These interviews did not contain any counterevidence; however, there was not as much specific content supporting the second hypotheses. Although there was discussion of a variety of methods, there were fewer ties to success/failure.
Small co.-operations product Manager (support) Sr. developer (strong support) Developer (support) Jr. developer (N/A)	↑ Uncertainty (plan-driven → ↓ match) and (controlled-flexible → ↑ match). The group provided support for H2. In the case of the jr. developer, no support was shown because only ad hoc comparison projects were mentioned.

back. ...She missed quite a few things. And based upon that...we had...the release went into production twice but had to be pulled back both times.

One implication is that a person can know how to do the ad hoc method. Another interviewee also discussed success with an ad hoc method. This second subject reported the use of many structures, such as regular feedback sessions, milestones, and other forms of scope boundaries and feedback initiatives. These were voluntary (self-imposed) controls. Because these were voluntary controls, the success was not replicable. We can fix this by training and requiring the use of the controls; however, once these controls become institutionalized, the method is no longer ad hoc—it becomes a controlled-flexible approach.

Several projects that were primarily ad hoc still had a few controls. They used multiple iterations and had significant feedback. However, acting with partial controls can be inefficient:

...things [were] constantly changing. [the customer] Didn't really like the placement of certain things. And literally, adding in new functionality all together. And this was on a fixed cost project. So, that didn't work in too well, especially when you're doing it for a fixed cost.

In this case, scope boundary controls were missing. The only control was a feedback process. There was no limit on customer requests and the project was allowed to run up extra costs. This is just one example that illustrates the importance of controls that work together and reinforce each other to build to a common goal.

In addition to observations on specific projects, most interviewees expressed a general opinion that a controlled-flexible approach was superior to an ad hoc approach.

I go back to an ad hoc as more of an R&D type of approach. It's very unpredictable. That you have to realize that nine out of ten times you're going to throw it away because it won't match anything of what a customer is going to want, but you've had a great time playing. You've learned a lot. But, when you come out...when you go into a controlled-flexible method you have a general idea, more like an original idea of what you're trying to build, but you have approaches to adapt and more of...as the market's changing, as you're going through this time, you know, because the

market doesn't stand still and so, you know, it matches quite a bit of what I've seen over the last three years that I've been working in this environment.

The evidence that supports H3—in uncertain environments, using controlled-flexible methods result in software products that are a better match to the market than using an ad hoc approach—is shown in Table 10. The data also point out the danger of incomplete sets of controls. One or two isolated control mechanisms may increase the product-market match at the cost of excessive cost overruns or other problems. Isolated controls that do not reinforce each other sufficiently can result in an out-of-control project.

5.4. Revised Research Model

Thorough analyses of the interview data support the three proposed hypotheses. One of the biggest differences from our original model is the recognition that the choice of development methods does not represent a selection among three discrete choices, but instead represents a range of possibilities involving trade-offs between control mechanisms. Another key discovery is the presence of two boundary conditions: time pressure and size of project. Therefore, we present a revised research model in Figure 5. This model changes the picture for development methods from three discrete boxes to one box to account for the opportunity for trade-offs among the types of methods and to reflect the boundary conditions.

6. Research Implications for Distributed Software Development

The study results can be used to understand and adapt controlled-flexible development approaches to various environments. In summary, the key findings are that there are two categories of emergent outcome controls: scope boundaries and dynamic feedback. Furthermore, it is essential to understand the interdependencies and trade-offs among the flexible control choices. For example, as the set of boundary controls restrict the solution set and reduce the choices the developer needs to make, there will be fewer requirements for ongoing feedback. The corollary of the previous statement is that, as the boundaries loosen and the set of options increase, feedback mechanisms will need to increase and become more pervasive.

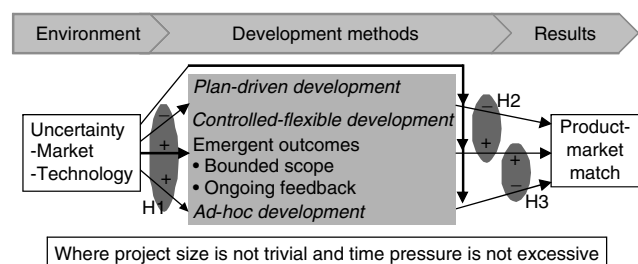
Table 10 Support for H3

ASP-custom Manager and lead (support) Sr. developer (strong support) Developer (support?)	↑ Uncertainty (controlled-flexible → better match than ad hoc) In general there was strong evidence of support for H3. The one case of nonsupport involved an ad hoc project that did not match the customer's needs. However, match was achieved through multiple iterations and trials. Therefore, it was not a pure ad hoc project. This controlled-flexible type of control was the basis of the success. Furthermore, the transcript revealed that this lone control was inefficient because it led to extra costs because of higher rework.
ASP-reporting Lead (strong support) Lead II (strong support) Lead III (N/A)	↑ Uncertainty + ad hoc → ↓ match. The transcripts for this group revealed solid evidence of the mismatch of an ad hoc approach. The lead developer (III) did not have ad hoc experience, so their transcript did not show evidence for or against the hypothesis. <i>Alternate evidence:</i> There was evidence of that a developer with more training could have more success with ad hoc. It implied that a trained developer could apply self-imposed controls, even if no external controls exist.
ASP-maintenance Director (support) Sr. manager (support) Sr. developer (support) Developer (weak support)	↑ Uncertainty (controlled-flexible → better match than ad hoc) These transcripts revealed consistent support for H3. Lower confidence was indicated for instances where the evidence more ambiguous. One interview stressed the importance of training to achieve proper self-control.
ASP-consumer Director (N/A) Manager (support) Tech lead (weak support) Developer (weak support)	Weak support for the H3. There were fewer examples involving ad hoc evidence in this set of interviews. In general, the hypothesis was supported. One new issue with an ad hoc approach was mentioned. One developer had worked supporting a sales staff that changed their minds frequently. Often, this developer would finish a project, and sales would no longer need the project. It may have been a problem that the stakeholders did not have to invest any effort in the development.
Small co.-operations product Manager (strong support) Sr. developer (strong support) Developer (support) Jr. developer (N/A)	↑ Uncertainty + ad hoc → ↓ match. Evidence of an inverse relationship between ad hoc and the product-market match. <i>Alternate Evidence:</i> These interviews clearly showed an experienced developer coping with an ad hoc approach and demonstrated the additional problems that an inexperienced developer may have in the same situation.

As an example of the utility of these results, consider their application to a distributed software development environment. In distributed software development, there are special communication challenges. Cultural and language barriers can impede communications. Organizational barriers can create problems if the developers have different organizational priorities than the stakeholders. If developers and stakeholders are in widely divergent time zones, communications may become asynchronous with 24-hours lapsing between each complete communication cycle (Cummings et al. 2009). Of course, feedback is simply a form of communication on the emergent outcome and anything that impairs communication will also impair the quality and usefulness of the feedback.

If we do not find some way to address these communication challenges, management's ability to use a controlled-flexible approach will be impaired. One potential solution is to make technical and social accommodations (technology-based agility and linkage agility—Sarker and Sarker 2009). For example, one of our interviewees from the Mobile Device Software company managed a team of off-shore

developers in Russia from his office in the United States. This manager used an extremely flexible approach with only broad scope limits. However, he had made special accommodations to support real time communications with his developers. The manager shifted his work day to wake up around 4 A.M. so that he could overlap with his development team in real time. While the team was at work the manager used instant messaging technology to remain constantly available for consultation and feedback. To break down barriers, he also made it a point to hold social conversations. He knew the birthdays and weddings of development team members and he made an

Figure 5 Revised Research Model

effort to send flowers and gifts for special occasions. These accommodations can help close the communications gaps that exist in a distributed environment.

In another interview, the developer was contracted to integrate an international customer into a service bureau. The customer and the developers were in different countries and there were language differences. However, the developer's own operations team was the expert in the field. The operations team became a local customer surrogate that managed development.

These two examples point to extraordinary efforts to improve feedback controls in a distributed environment. In the first example, technology and social accommodations were made. In the second example, a local customer surrogate was tapped to provide project feedback.

An alternative to improving the feedback channel would be to minimize the need for feedback. As noted earlier, flexible approaches rely on a portfolio of controls and control substitution can occur. If we can lock down the scope boundaries we can minimize the amount of feedback required. At one extreme, we could lock down the scope so tightly that we are using a plan-driven approach. However, our research shows that this extreme form of specification will not be possible in conditions of uncertainty. A possible solution is a mixing of development methods within a project (Boehm and Turner 2004). It may be possible to manage part of a development using a plan-driven approach and another part using a flexible approach. This would involve a modular development approach that isolates the areas of uncertainty into specific modules. The total amount of feedback required would thus be reduced because only the uncertain modules would require detailed feedback. Even within these flexible modules, the need for feedback can be minimized by tightening the scope as much as feasible.

In summary, a controlled-flexible approach in a globally distributed environment faces special challenges because of communications issues. To accommodate the need for flexibility our study would suggest three adjustments: (1) only use a flexible approach on the portions of the system that are most affected by uncertainty. (2) Within modules where flexibility is required, reduce the feedback

required by tightening scope boundaries as much as is feasible. (3) Make adaptations to improve feedback communications.

7. Research Limitations

This research is qualitative with data from structured interviews. Generalization of the results through a survey-based approach would be a logical next step. However, development of a survey instrument may not be straightforward. A qualitative investigation allows researchers to probe stated practices to determine if the practice is faithfully appropriated. A survey instrument needs a clear test to determine the agility of a practice. This can be accomplished through using a taxonomy that defines when a practice is agile (Conboy 2009).

Furthermore, the current study does not claim to have found the "best" development method. This study provides tools for evaluating development methods by focusing on types of control and does not speak to specific controls. The research concentrates on conditions of uncertainty. Other control priorities may apply in other conditions, such as extreme time pressure. This suggests a portfolio of control choices and suggests that selection of a control set depends on the management priorities of the project at hand. In one case the priority might be uncertainty, in another case time pressure, in a third case system reliability, and so on. In this regard, we call for further research into the conditions that shape the choice of methods used. We also recognize that conditions may not be clearly separable. For example, a project may require extreme reliability and may have uncertainty. Therefore, future research needs to address trade-offs in complex situations.

8. Research Contributions

This study provides a rich understanding of the nature of control when using flexible software development approaches. Based on an in-depth analysis of software development methods presented in (Harris et al. 2009), we describe a new category of controls: emergent outcome controls which are particularly relevant to the use of flexible methods. That study demonstrated that current methods-in-use use a portfolio of controls that vary by project, within

project, and by development team; so that rather than speaking of mutually exclusive, discrete development approaches, it is more appropriate to discuss how individual controls are used to produce approaches that may be termed ad hoc, plan-driven, or controlled-flexible. The results of the research in this paper show that increased uncertainty of the software product market and the technology will lead to the use of more flexible approaches and the use of more controlled-flexible development methods results in a better product-market match.

The theoretical implications of this research are that the concept of emergent outcome controls elaborates on and extends control theory in a way that is important to the study of today's dynamic work environments. Our results support and extend previous research on control theory in software development that models a portfolio of controls, rather than singular, discrete controls. We explain that a portfolio approach is necessary when individual controls are not sufficient. In addition, the study identifies two key boundary conditions (time pressure and project size) that influence the relationships between uncertain environments and flexible work methods in the dynamic capabilities extension of the resource-based view of the firm.

The practical implications are that project managers can use this understanding of how and why flexible approaches are used in practice to guide their selection of software development methods and of a control portfolio. This is particularly important in today's complex development environment where the exercise of control cannot easily be done through direct, face-to-face observation or interaction with team members who may be distributed in different countries or companies, or both. Many project managers must meet the challenge of managing projects in uncertain environments that argue for increased flexibility, while operating in a distributed, virtual control setting. These project managers can use the study results to adapt development approaches that control flexibility through emergent outcome controls.

Acknowledgments

The authors thank the special issue editors, the anonymous reviewers, and the special issue workshop participants for their insightful comments on this paper.

References

- Auerback, C., L. Silverstein. 2003. *Qualitative Data: An Introduction to Coding and Analysis*. New York: University Press, New York.
- Austin, R., L. Devin. 2003. *Artful Making: What Managers Need to Know About How Artists Work*. FT Prentice-Hall, Upper Saddle River, NJ.
- Barney, J. 1996. The resource-based theory of the firm. *Organ. Sci.* 7(5) 469.
- Baskerville, R., J. Stage. 1996. Controlling prototype development through risk analysis. *MIS Quart.* 20(4) 481–504.
- Baskerville, R., L. Levine, J. Pries-Heje, B. Ramesh, S. Slaughter. 2001. How Internet software companies negotiate quality. *IEEE Comput.* 34(5) 51–57.
- Bhattacharya, S., V. Krishnan, V. Mahajan. 1998. Managing new product definition in highly dynamic environments. *Management Sci.* 44(11) 50–64.
- Beck, K., C. Andres. 2004. *Extreme Programming Explained: Embrace Change*, 2nd ed. Addison-Wesley, Boston.
- Beck, K., M. Beedle, A. Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick. 2001. The agile manifesto. Retrieved February 2005, <http://www.agilemanifesto.org>.
- Boehm, B., R. Turner. 2004. *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley, Boston.
- Bourgeois, L., III, K. Eisenhardt. 1988. Strategic decision processes in high velocity environments: Four cases in the microcomputer industry. *Management Sci.* 34(7) 816–835.
- Bowman, J. 2008. Performance art. Retrieved January 2008, <http://www.bright.net/~dapoets/performa.htm>.
- Brown, S., K. Eisenhardt. 1997. The art of continuous change: Linking complexity theory and time-paced evolution in relentlessly shifting organizations. *Admin. Sci. Quart.* 42(1) 1–34.
- Cardinal, L., S. Sitkin, C. Long. 2004. Balancing and rebalancing in the creation and evolution of organizational control. *Organ. Sci.* 15(4) 411–431.
- Choudhury, V., R. Sabherwal. 2003. Portfolios of control in outsourced software development projects. *Inform. Systems Res.* 14(3) 291–314.
- Conboy, K. 2009. Agility from first principles: Reconstructing the concept of agility in information systems development. *Inform. Systems Res.* 20(3) 329–354.
- Cummings, J., J. Espinosa, C. Pickering. 2009. Crossing spatial and temporal boundaries in globally distributed projects: A relational model of coordination delay. *Inform. Systems Res.* 20(3) 420–439.
- Cusumano, M., D. Yoffie. 1999. Software development on Internet time. *IEEE Comput.* 32(10) 60–69.
- Dahan, E., H. Mendelson. 2001. An extreme-value model of concept testing. *Management Sci.* 47(1) 102–116.
- Dubé, L., G. Paré. 2003. Rigor in information systems positivist case research: Current practices, trends, and recommendations. *MIS Quart.* 27(4) 597–636.
- Eisenhardt, K. 1989a. Making fast strategic decisions in high velocity environments. *Acad. Management J.* 32(3) 543–576.
- Eisenhardt, K. 1989b. Building theories from case study research. *Acad. Management Rev.* 14(4) 532–550.
- Eisenhardt, K., J. Martin. 2000. Dynamic capabilities: What are they? *Strategic Management J.* 21(10–11) 1105–1121.

- Eisenhardt, K., B. Tabrizi. 1995. Accelerating adaptive processes: Product innovation in the global computer industry. *Admin. Sci. Quart.* **40**(1) 84–110.
- Eisenhardt, K. M., D. N. Sull. 2001. Strategy as simple rules. *Harvard Bus. Rev.* **79** (1) 106–116.
- Fitzgerald, B., G. Hartnett, K. Conboy. 2006. Customizing agile methods to software practices at Intel Shannon. *Eur. J. Inform. Systems* **15**(2) 200–213.
- Fitzgerald, B., N. Russo, T. O’Kane. 2003. Software development tailoring at Motorola. *Comm. ACM* **46**(4) 65–70.
- Harris, M., K. Aebischer, T. Klaus. 2007. The whitewater process: Software product development in small IT businesses. *Comm. ACM* **50**(5) 89–93.
- Harris, M., A. Hevner, R. Collins. 2009. Controls in flexible software development. *Comm. Assoc. Inform. Systems* **24**, Article 43.
- Henderson, J., L. Soonchul. 1992. Managing I/S design teams: A control theories perspective. *Management Sci.* **38**(6) 757–777.
- Huber, G., A. Van de Ven. 1995. *Longitudinal Field Research Methods: Studying Processes in Organizational Change*. Sage, London.
- Iansiti, M. 1995. Shooting the rapids: Managing product development in turbulent environments. *California Management Rev.* **38**(1) 37–58.
- Jap, S. D. 2001. Perspectives on joint competitive advantage in buyer-supplier relationships. *Internat. J. Res. Marketing* **18**(1) 19–35.
- Karagozoglu, N., W. B. Brown. 1993. Time-based management of the new product development process. *J. Product Innovation Management* **10**(3) 204–215.
- Kirsch, L. 1997. Portfolios of control modes and IS project management? *Inform. Systems Res.* **8**(3) 215–239.
- Kirsch, L. 2004. Deploying common systems globally: The dynamics of control. *Inform. Systems Res.* **15**(4) 374–395.
- Krishnan, V., K. Ulrich. 2001. Product development decisions: A review of the literature. *Management Sci.* **47**(1) 1–21.
- Kruchten, P. 2000. *The Rational Unified Process: An Introduction*, 2nd ed. Addison-Wesley, Reading, MA.
- Lyytinen, K., G. Rose. 2006. Information system development agility as organizational learning. *Eur. J. Inform. Systems* **15**(2) 183–199.
- Lyytinen, K., L. Mathiassen, J. Ropponen. 1998. Attention shaping and software risk—A categorical analysis of four classical risk management approaches. *Inform. Systems Res.* **9**(3) 233–255.
- MacCormack, A., R. Verganti. 2003. Managing the sources of uncertainty: Matching process and context in software development. *J. Product Innovation Management* **20**(3) 217–232.
- McConnell, S. 1996. *Rapid Development*. Microsoft Press, Redmond, WA.
- Mintzberg, H., A. McHugh. 1985. Strategy formation in an adhoc-racy. *Admin. Sci. Quart.* **30** 160–197.
- Moorman, C., A. Miner. 1998. Organizational improvisation and organizational memory. *Acad. Management Rev.* **23**(4) 698–723.
- Nambisan, S. 2003. Information systems as a reference discipline for new product development. *MIS Quart.* **27**(1) 1–18.
- Nambisan, S., D. Wilemon. 2000. Software development and new product development: Potentials for cross-domain knowledge sharing. *IEEE Trans. Engrg. Management* **47**(2) 211–220.
- Nidumolu, S., M. Subramani. 2003–2004. The matrix of control: Combining process and structure approaches to managing software development. *J. Management Inform. Systems* **20**(3) 159–196.
- O’Reilly, C. A., M. L. Tushman. 2008. Ambidexterity as a dynamic capability: Resolving the innovator’s dilemma. *Res. Organ. Behav.* **28** 185–206.
- Orlikowski, W. 1991. Integrated information environment or matrix of control? The contradictory implications of information technology. *Accounting Management Inform. Tech.* **1**(1) 9–42.
- Ouchi, W. 1977. The relationship between organizational structure and organizational control. *Admin. Sci. Quart.* **22**(1) 95–113.
- Ouchi, W. 1979. A conceptual framework for the design of organizational control mechanisms. *Management Sci.* **25**(9) 833–848.
- Ouchi, W. 1980. Markets, bureaucracies, and clans. *Admin. Sci. Quart.* **25**(1) 129–141.
- Ouchi, W., J. Johnson. 1978. Types of organizational control and their relationship to emotional well being. *Admin. Sci. Quart.* **23**(2) 293–317.
- Pavlou, P. A., O. A. El Sawy. 2006. From IT leveraging competence to competitive advantage in turbulent environments: The case of new product development. *Inform. Systems Res.* **17**(3) 198–227.
- Peters, T., R. Waterman. 1982. *In Search of Excellence: Lessons from America’s Best-Run Companies*, 1st ed. Harper & Row, New York.
- Robey, D. 1996. *Designing Organizations*. Irwin, Homewood, IL.
- Sarker, S., S. Sarker. 2009. Exploring agility in distributed information systems development (ISD) teams: An interpretive study in an offshoring context. *Inform. Systems Res.* **20**(3) 440–461.
- Schwaber, K., M. Beedle. 2002. *Agile Software Development with Scrum*. Prentice-Hall, Upper Saddle River, NJ.
- Teece, D., G. Pasano, A. Shuen. 1997. Dynamic capabilities and strategic management. *Strategic Management J.* **18**(7) 509–533.
- Takeuchi, H., I. Nonaka. 1986. The new new product development game. *Harvard Bus. Rev.* **64**(1) 137–146.
- Tripsas, M., G. Gavetti. 2000. Capabilities, cognition, and inertia: Evidence from digital imaging. *Strategic Management J.* **21**(10/11) 1147–1161.
- Verona, G., D. Ravasi. 2003. Unbundling dynamic capabilities: An exploratory study of continuous product innovation. *Indust. Corporate Change* **12**(3) 577–606.
- Weick, K. 1998. Improvisation as a mindset for organizational analysis. *Organ. Sci.* **9**(5) 543–555.
- Wheeler, B. 2002. NEBIC: A dynamic capabilities theory for assessing net-enablement. *Inform. Systems Res.* **13**(2) 125–146.
- Yin, R. 1994. *Case Study Research: Design and Methods*, 2nd ed. Sage Publications, Inc., Thousand Oaks, CA.